

Git Challenges

1. Resolve Merge Conflicts

- Create a merge conflict intentionally (two users editing the same line).
- Resolve the conflict and push the changes.

Objective

To create and resolve a merge conflict when two branches modify the same line of a file.

Purpose

To understand how Git handles conflicting changes made by different developers.

Implementation

Step 1: Create a Git repository

```
mkdir git-merge-conflict  
cd git-merge-conflict  
git init
```

Create a file:

```
echo "Hello from Asifshaik" > msg.txt
```

Check:

```
cat msg.txt
```

```
git add msg.txt  
git commit -m "Initial commit"
```

Step 2: Create Developer 1 branch

```
git checkout -b developer1
```

Edit the file:

vi msg.txt

Change:

Hello from Git

to:

Hello from techie

Commit:

git add msg.txt

git commit -m "Updated message by Developer 1"

Step 3: Create Developer 2 branch

Go back to the original branch:

git checkout master

Create another branch:

git checkout -b developer2

Edit the same line:

vi msg.txt

Change:

Hello from Git

to:

Hello from horizon

Save and exit.

Commit:

git add msg.txt

git commit -m "Updated message by Developer 2"

4. Implementation — Create the Conflict

Now you have:

Developer 1:

Hello from techie

Developer 2:

Hello from horizon

Both developers changed the same line.

You are on developer2, so merge Developer 1:

```
git merge developer1
```

Step 5: View the conflict

```
cat msg.txt
```

You will see:

```
<<<<<<< HEAD
Hello from horizon
=====
Hello from techie
>>>>>>> developer1
```

Step 6: Stage the resolved file

```
git add msg.txt
```

Check:

```
git status
```

You should see that the conflict has been resolved and Git is ready to complete the merge.

Step 7: Complete the merge

```
git commit -m "Resolved merge conflict"
```

Check the history:

```
git log --oneline --graph --all
```

You should see a merge structure similar to:

```
* abc1234 Resolved merge conflict
|\
| * c9f7242 Updated message by Developer 1
* | a965028 Updated message by Developer 2
|/
* a7d8036 Initial commit
```

6. Push the Changes to GitHub

If your repository has a GitHub remote:

```
git remote -v
```

Then push the branch:

```
git push -u origin developer2
```

If you want the resolved merge on main instead, you can merge it into main and push:

```
git checkout master
git merge developer2
git push origin master
```

Example

Initial version

Hello from Git

Developer 1 changes it

Hello from techie

Developer 2 changes the same line

Hello from horizon

Git creates the conflict

<<<<<<< HEAD

Hello from horizon

=====

Hello from techie

>>>>>>> developer1

Final resolved version

Hello from horizon and techie

```
asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ touch msg.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ vi msg.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ cat msg.txt
Hello from Asifshaik

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ gid add msg.txt
bash: gid: command not found

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git add msg.txt
warning: in the working copy of 'msg.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git commit -m "initial commit"
[master (root-commit) a7d8036] initial commit
1 file changed, 1 insertion(+)
create mode 100644 msg.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ pwd
/c/Users/asifs/git-merge-conflict

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git status
On branch master
nothing to commit, working tree clean

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git log --oneline
a7d8036 (HEAD -> master) initial commit

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git checkout -b developer1
Switched to a new branch 'developer1'

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer1)
$ git branch
* developer1
  master
```

```

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer1)
$ vi msg.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer1)
$ cat msg.txt
Hello from techie

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer1)
$ git add msg.txt
warning: in the working copy of 'msg.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer1)
$ git commit -m "Updated message by techie"
[developer1 c9f7242] Updated message by techie
1 file changed, 1 insertion(+), 1 deletion(-)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer1)
$ git branch
* developer1
  master

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer1)
$ git log --oneline --all
c9f7242 (HEAD -> developer1) Updated message by techie
a7d8036 (master) initial commit

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer1)
$ git checkout master
Switched to branch 'master'

```

```

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2)
$ vi msg.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2)
$ git add msg.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2)
$ git commit -m "Updated message by horizon"
[developer2 a965028] Updated message by horizon
1 file changed, 1 insertion(+), 1 deletion(-)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2)
$ git log --oneline --all --graph
* a965028 (HEAD -> developer2) Updated message by horizon
| * c9f7242 (developer1) Updated message by techie
|/
* a7d8036 (master) initial commit

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2)
$ git merge developer1
Auto-merging msg.txt
CONFLICT (content): Merge conflict in msg.txt
Automatic merge failed; fix conflicts and then commit the result.

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2|MERGING)
$ cat msg.txt
<<<<<< HEAD
Hello from horizon
=====
Hello from techie
>>>>>> developer1

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2|MERGING)
$ git merge developer1
error: Merging is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <files>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2|MERGING)
$ cat message.txt
cat: message.txt: No such file or directory

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2|MERGING)
$ cat msg.txt
<<<<<< HEAD
Hello from horizon
=====
Hello from techie
>>>>>> developer1

```

```

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2|MERGING)
$ vi msg.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2|MERGING)
$ cat msg.txt
Hello from horizon and techie

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2|MERGING)
$ git add msg.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2|MERGING)
$ git status
On branch developer2
All conflicts fixed but you are still merging.
(use "git commit" to conclude merge)

Changes to be committed:
  modified:   msg.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2|MERGING)
$ git commit -m "Resolved merge conflict"
[developer2 503ab68] Resolved merge conflict

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2)
$ git log --oneline --graph --all
* 503ab68 (HEAD -> developer2) Resolved merge conflict
|
| * c9f7242 (developer1) Updated message by techie
| * a965028 Updated message by horizon
|/
* a7d8036 (master) initial commit

```

7. Conclusion

In this challenge, a merge conflict was intentionally created by having two developers modify the same line differently in separate branches. Git could not automatically determine which change to keep, so it marked the file with conflict markers. The conflict was resolved manually by selecting the required content, staging the file, and creating a merge commit. Finally, the resolved branch was pushed to GitHub.

2. Recover Deleted Branch

Delete a local branch and then recover it using the reflog.

1. Purpose

The purpose of this challenge is to learn how to **recover a deleted local Git branch using git reflog**.

git reflog records the previous movements of HEAD and branch references, so it can help us find the commit where the deleted branch was pointing.

2. Objective

- Create a new Git branch.
- Make a commit on that branch.
- Delete the branch locally.
- Use git reflog to find the deleted branch's commit.

- Recreate the branch using that commit.
- Verify that the branch has been successfully recovered.

Steps / Implementation

Step 1: Go to your Git repository

```
cd ~/git-merge-conflict
```

Check the repository:

```
git status
```

Step 2: Create a new branch

First switch to master:

```
git checkout master
```

Create a branch called recovery-test:

```
git checkout -b recovery-test
```

Check:

```
git branch
```

Step 3: Create a file on the new branch

```
echo "Important recovery data" > recovery.txt
```

Check the file:

```
cat recovery.txt
```

Step 4: Commit the file

```
git add recovery.txt
```

Then:

```
git commit -m "Added recovery data"
```

Step 5: Check the commit

```
git log --oneline --all
```

Step 6: Switch to master

You cannot delete the branch you are currently using, so first switch to master:

```
git checkout master
```

Step 7: Delete the local branch

Run:

```
git branch -d recovery-test
```

5. Recover the Deleted Branch Using Reflog

Step 8: Check the reflog

Run:

```
git reflog
```

You may see:

```
abc1234 HEAD@{1}: commit: Added recovery data
```

```
a7d8036 HEAD@{2}: checkout: moving from recovery-test to master
```

Find the commit containing:

```
commit: Added recovery data
```

Copy its commit ID.

For example:

```
abc1234
```

Step 9: Recreate the deleted branch

Use the commit ID:

```
git checkout -b recovery-test abc1234
```

Replace abc1234 with your actual commit ID.

6. Verify the Recovery

Check the branches:

```
git branch
```

You should see:

```
* recovery-test
master
```

Check the commit:

```
git log --oneline -3
```

```
asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (developer2)
$ git checkout master
Switched to branch 'master'

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git checkout -b recovery-test
Switched to a new branch 'recovery-test'

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (recovery-test)
$ git branch
  developer1
  developer2
* master
* recovery-test

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (recovery-test)
$ touch recovery_data

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (recovery-test)
$ vi recovery_data

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (recovery-test)
$ git add recovery.txt
fatal: pathspec 'recovery.txt' did not match any files

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (recovery-test)
$ git add recovery_data
warning: in the working copy of 'recovery_data', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (recovery-test)
$ git commit -m "Added recovery data"
[recovery-test 2a3c8ac] Added recovery data
1 file changed, 1 insertion(+)
create mode 100644 recovery_data

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (recovery-test)
$ git log --oneline --all
2a3c8ac (HEAD -> recovery-test) Added recovery data
503ab68 (developer2) Resolved merge conflict
a965028 Updated message by horizon
c9f7242 (developer1) Updated message by techie
a7d8036 (master) initial commit

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (recovery-test)
$ git checkout master
Switched to branch 'master'

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git branch -d recovery-test
error: The branch 'recovery-test' is not fully merged.
If you are sure you want to delete it, run 'git branch -D recovery-test'.

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git branch -d recovery_data
error: branch 'recovery_data' not found.
```

```

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git branch -D recovery-test
Deleted branch recovery-test (was 2a3c8ac).

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git reflog
a7d8036 (HEAD -> master) HEAD@{0}: checkout: moving from recovery-test to master
2a3c8ac HEAD@{1}: commit: Added recovery data
a7d8036 (HEAD -> master) HEAD@{2}: checkout: moving from master to recovery-test
a7d8036 (HEAD -> master) HEAD@{3}: checkout: moving from developer2 to master
503ab68 (developer2) HEAD@{4}: commit (merge): Resolved merge conflict
a965028 HEAD@{5}: commit: Updated message by horizon
a7d8036 (HEAD -> master) HEAD@{6}: checkout: moving from master to developer2
a7d8036 (HEAD -> master) HEAD@{7}: checkout: moving from developer1 to master
c9f7242 (developer1) HEAD@{8}: commit: Updated message by techie
a7d8036 (HEAD -> master) HEAD@{9}: checkout: moving from master to developer1
a7d8036 (HEAD -> master) HEAD@{10}: commit (initial): initial commit

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git checkout -b recovery-test ^C

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git checkout -b recovery-test 2a3c8ac
Switched to a new branch 'recovery-test'

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (recovery-test)
$ git branch
  developer1
  developer2
  master
* recovery-test

```

Conclusion

In this challenge, a local Git branch named recovery-test was created and a commit was made on it. The branch was then intentionally deleted. Using git reflog, we located the commit that the deleted branch previously pointed to and recreated the branch from that commit.

3. Undo Wrong Push

Push a wrong commit to GitHub, then undo it without losing history.

1. Purpose

The purpose of this challenge is to learn how to undo an incorrect commit that has already been pushed to GitHub without deleting the existing history.

For this task, we will use git revert instead of git reset.

2. Objective

- Create a wrong change in a Git repository.

- Commit the wrong change.
- Push it to GitHub.
- Identify the wrong commit.
- Use git revert to undo the change.
- Push the revert commit to GitHub.
- Verify that the original commit remains in Git history.

3. Steps / Implementation

Step 1: Go to your Git repository

For example, use your existing repository:

```
cd ~/git-merge-conflict
```

```
git status
```

Step 2: Create a wrong change

Let's create a file called:

```
wrong.txt
```

Run:

```
echo "This is a wrong change" > wrong.txt
```

Step 3: Commit the wrong change

Stage the file:

```
git add wrong.txt
```

Commit it:

```
git commit -m "Added wrong file"
```

4. Push the Wrong Commit to GitHub

Push your branch:

```
git push origin master
```

5. Identify the Wrong Commit

Check the history:

```
git log --oneline -3
```

6. Undo the Wrong Commit

Now use:

```
git revert abc1234
```

Replace abc1234 with your actual wrong commit ID.

Git will create a new commit that reverses the changes made by the wrong commit.

You may see an editor open with a message like:

```
Revert "Added wrong file"
```

7. Check the History

Run:

```
git log --oneline -4
```

8. Verify the File

Run:

```
Ls  
git status
```

9. Push the Revert to GitHub

Now push the new revert commit:

```
git push origin master3
```

```

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (recovery-test)
$ git switch master
Switched to branch 'master'

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ touch wrong.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ vi wrong.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git add wrong.txt
warning: in the working copy of 'wrong.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git commit -m "Added wrong file"
[master ec7ccbc] Added wrong file
1 file changed, 1 insertion(+)
create mode 100644 wrong.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git push origin master
fatal: 'origin' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git remote -v

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git remote add origin git@github.com:asifshaik6558/git-undo-wrong-push.git

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git push -u origin master
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (6/6), 493 bytes | 123.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:asifshaik6558/git-undo-wrong-push.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git revert ec7ccbc
[master fd1e79e] Revert "Added wrong file"
1 file changed, 1 deletion(-)
delete mode 100644 wrong.txt

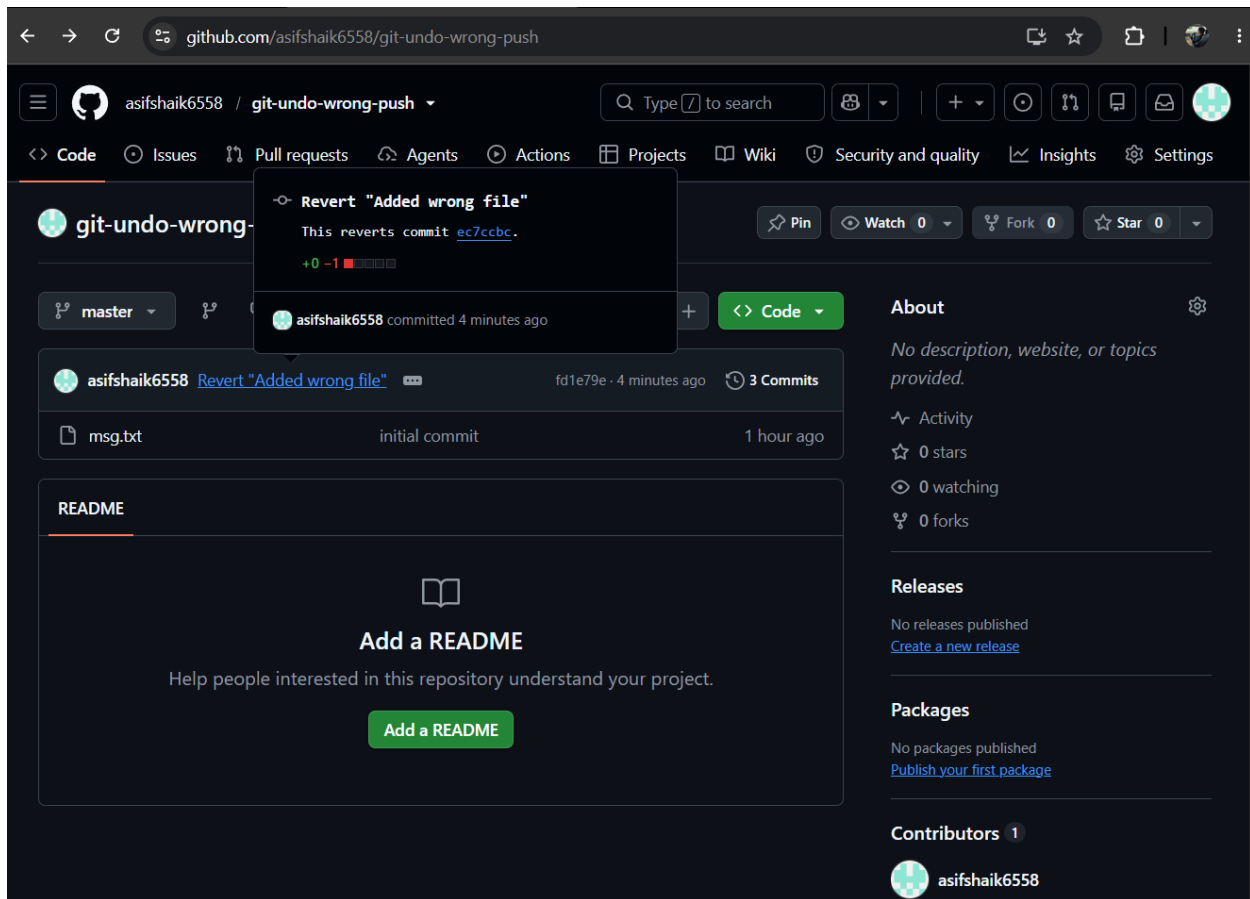
asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git revert ec7ccbc
[master fd1e79e] Revert "Added wrong file"
1 file changed, 1 deletion(-)
delete mode 100644 wrong.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git log --oneline -3
fd1e79e (HEAD -> master) Revert "Added wrong file"
ec7ccbc (origin/master) Added wrong file
a7d8036 initial commit

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ ls
msg.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git push origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (1/1), done.
Writing objects: 100% (2/2), 262 bytes | 65.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:asifshaik6558/git-undo-wrong-push.git
ec7ccbc..fd1e79e master -> master

```



13. Conclusion

In this challenge, a wrong commit was intentionally created and pushed to GitHub. Instead of deleting or rewriting the existing history, `git revert` was used to create a new commit that reverses the wrong changes. The original wrong commit remains visible in the Git history, providing a clear record of what happened.

4. Amend a Commit

Make a commit, then add a missing file to it using `git commit --amend`.

1. Purpose

The purpose of this challenge is to learn how to **modify the most recent Git commit** using `git commit --amend`.

This is useful when you forgot to include a file or want to correct the commit message **before sharing the commit with others**.

2. Objective

- Create a file and commit it.
- Realize that another file was missing from the commit.
- Create the missing file.
- Add it to the staging area.
- Use git commit --amend to add the missing file to the previous commit.
- Verify that both files are now part of **one commit**.

3. Steps / Implementation

Step 1: Check your repository

```
cd ~/git-merge-conflict
```

Check:

```
git status
```

Step 2: Create the first file

Run:

```
echo "Application is running" > app.txt
```

Check:

```
cat app.txt
```

Step 3: Commit only the first file

```
git add app.txt
```

Then:

```
git commit -m "Added application files"
```

4. Create the Missing File

Now create the file that we forgot:

```
echo "PORT=8080" > config.txt
```

Check:

```
cat config.txt
```

Check Git:

```
git status
```

5. Add the Missing File to the Previous Commit

First stage the missing file:

```
git add config.txt
```

Check:

```
git status
```

Now use:

```
git commit --amend --no-edit
```

6. Verify the Amended Commit

Run:

```
git log --oneline -3
```

Now check which files are in that commit:

```
git show --stat HEAD
```

7. Verify the Files

Run:

```
Ls
```

```

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ vi app.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git add app.txt
warning: in the working copy of 'app.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git commit -m "Added application files"
[master 5c7d67c] Added application files
1 file changed, 1 insertion(+)
create mode 100644 app.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ vi config.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ vi config.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git status
On branch master
Your branch is ahead of 'origin/master' by 1 commit.
(use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    config.txt
    congig.txt

nothing added to commit but untracked files present (use "git add" to track)

```

```

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git add config.txt
warning: in the working copy of 'config.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git status
On branch master
Your branch is ahead of 'origin/master' by 1 commit.
(use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   config.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    congig.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git commit --amend --no-edit
[master f049ca2] Added application files
Date: Sun Aug 23 16:56:42 2026 +0530
2 files changed, 2 insertions(+)
create mode 100644 app.txt
create mode 100644 config.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git log --oneline -3
f049ca2 (HEAD -> master) Added application files
fd1e79e (origin/master) Revert "Added wrong file"
ec7ccbc Added wrong file

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git show --stat HEAD
commit f049ca2cfab32f41f2e88d07114e20ff296c0be7 (HEAD -> master)
Author: Asif <asifshaik6558@gmail.com>
Date: Sun Aug 23 16:56:42 2026 +0530

    Added application files

 app.txt | 1 +
 config.txt | 1 +
 2 files changed, 2 insertions(+)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ ls
app.txt  config.txt  congig.txt  msg.txt

```

10. Conclusion

In this challenge, a commit was created with only app.txt. After realizing that config.txt was missing, the file was staged and added to the previous commit using:

```
git commit --amend --no-edit
```

Instead of creating a second commit, Git updated the most recent commit so that it contained both files.

5. Cherry-pick a Commit

Take a specific commit from one branch and apply it to another branch.

1. Purpose

The purpose of this challenge is to learn how to take one specific commit from one branch and apply it to another branch using `git cherry-pick`.

This is useful when you need a particular fix or change from another branch without merging the entire branch.

2. Objective

- Create a separate branch.
- Make a specific commit on that branch.
- Switch to another branch.
- Use `git cherry-pick` to apply only that commit.
- Verify that the change is added to the target branch.

4. Steps / Implementation

Step 1: Go to your repository

Use your practice repository:

```
cd ~/git-merge-conflict
```

Check:

```
git status
```

Step 2: Create a feature branch

Switch to master:

```
git checkout master
```

Create a new branch:

```
git checkout -b feature-login
```

Check:

```
git branch
```

Step 3: Create a change on the feature branch

Create a file:

```
echo "Login feature added" > login.txt
```

Check:

```
cat login.txt
```

Step 4: Commit the change

```
git add login.txt
```

Then:

```
git commit -m "Added login feature"
```

6. Switch to the Target Branch

Now go back to master:

```
git checkout master
```

Check:

```
git branch
```

Now check whether login.txt exists:

```
Ls
```

7. Cherry-pick the Commit

Now run:

```
git cherry-pick abc1234
```

Difference Between Merge and Cherry-pick

Command	Purpose
git merge feature-login	Brings the branch's changes/history together

git cherry-pick <commit>	Takes only one specific commit
git rebase	Replays commits onto another base

```

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git checkout -b feature-login
Switched to a new branch 'feature-login'

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (feature-login)
$ git branch
  developer1
  developer2
* feature-login
  master
  recovery-test

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (feature-login)
$ touch login

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (feature-login)
$ vi login

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (feature-login)
$ git add login
warning: in the working copy of 'login', LF will be replaced by CRLF the next time Git touches

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (feature-login)
$ git commit -m "Added login feature"
[feature-login 2cca90b] Added login feature
 1 file changed, 1 insertion(+)
 create mode 100644 login

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (feature-login)
$ git checkout master
Switched to branch 'master'
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git branch
  developer1
  developer2
  feature-login
* master
  recovery-test

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ ls
app.txt  config.txt  congig.txt  msg.txt

```

```
asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git cherry-pick 2cca90b
[master 34c8747] Added login feature
Date: Sun Aug 23 17:09:16 2026 +0530
1 file changed, 1 insertion(+)
create mode 100644 login

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ ls
app.txt  config.txt  congig.txt  login  msg.txt
```

11. Conclusion

In this challenge, a specific commit was created on the feature-login branch. Instead of merging the entire branch, git cherry-pick was used to apply only that particular commit to master.

Key command

git cherry-pick <commit-id>

Key concept

Cherry-pick allows you to select and apply a specific commit from another branch without merging the entire branch.

6. Interactive Rebase

Reorder and squash multiple commits into a single clean commit.

1. Purpose

The purpose of this challenge is to learn how to use interactive rebase to clean up Git history.

With interactive rebase, you can:

- Reorder commits
- Squash multiple commits into one
- Edit commit messages
- Remove unwanted commits

In this challenge, we will create 3 commits and then squash them into one clean commit.

2. Objective

- Create a separate branch.
- Make three small commits.
- View the commit history.
- Use git rebase -i to combine the commits.
- Squash the commits into one.
- Verify the clean Git history.

4. Steps / Implementation

Step 1: Go to your repository

```
cd ~/git-merge-conflict
```

Check:

```
git status
```

Step 2: Create a new branch

Switch to master:

```
git checkout master
```

Create a branch:

```
git checkout -b rebase-demo
```

5. Create Three Commits

Step 3: Commit 1

Create a file:

```
echo "HTML content" > webpage.txt
```

Stage:

```
git add webpage.txt
```

Commit:

```
git commit -m "Added HTML"
```

Step 4: Commit 2

Add another line:

```
echo "CSS styling" >> webpage.txt
```

Check:

```
cat webpage.txt
```

Commit:

```
git add webpage.txt  
git commit -m "Added CSS"
```

Step 5: Commit 3

Add one more line:

```
echo "Footer section" >> webpage.txt
```

Commit:

```
git add webpage.txt  
git commit -m "Added footer"
```

. Check the History

Run:

```
git log --oneline --graph -4
```

7. Start Interactive Rebase

We want to modify the last 3 commits.

Run:

```
git rebase -i HEAD~3
```

```

MINGW64/c/Users/asifs/git-merge-conflict
asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git checkout -b rebase-demo
Switched to a new branch 'rebase-demo'

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ touch webpage.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ vi webpage.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git add webpage.txt
warning: in the working copy of 'webpage.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git commit -m "Added web page"
[rebase-demo b141591] Added web page
1 file changed, 1 insertion(+)
create mode 100644 webpage.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ vi webpage.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git add webpage.txt
warning: in the working copy of 'webpage.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git commit -m "Added CSS"
[rebase-demo 6ccb3d8] Added CSS
1 file changed, 1 insertion(+)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ vi webpage.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git add webpage.txt
warning: in the working copy of 'webpage.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git commit -m "Added footer"
[rebase-demo 039629b] Added footer
1 file changed, 1 insertion(+)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git log --oneline --graph -4
* 039629b (HEAD -> rebase-demo) Added footer
* 6ccb3d8 Added CSS
* b141591 Added web page
* 34c8747 (master) Added login feature

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git rebase -i HEAD~3
[detached HEAD ff70662] Added web page
Date: Sun Aug 23 18:02:08 2026 +0530
1 file changed, 3 insertions(+)
create mode 100644 webpage.txt
Successfully rebased and updated refs/heads/rebase-demo.

```

```

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git log --oneline --graph -4
* ff70662 (HEAD -> rebase-demo) Added web page
* 34c8747 (master) Added login feature
* f049ca2 Added application files
* fd1e79e (origin/master) Revert "Added wrong file"

```

15. Conclusion

In this challenge, three separate commits were created and then cleaned up using interactive rebase. The `git rebase -i HEAD~3` command allowed the commits to be selected and combined using squash. The final repository contained one clean commit instead of three small commits.

Key command

`git rebase -i HEAD~3`

Key concept

Interactive rebase is used to clean and organize Git history before sharing it with others.

7. Tagging & Release

Create a version tag (v1.0), push it to GitHub, then delete and restore it.

1. Purpose

The purpose of this challenge is to learn how to use Git tags to mark a specific commit as a version or release.

We will create a tag called v1.0, push it to GitHub, delete it locally and remotely, and then restore the tag.

2. Objective

- Create a version tag v1.0.
- Push the tag to GitHub.
- Verify the tag on GitHub.
- Delete the tag locally.
- Delete the tag from GitHub.
- Restore the deleted tag.
- Push the restored tag back to GitHub.

3. Implementation

We will use your current repository:

```
cd ~/git-merge-conflict
```

Step 1: Check your current commit

Run:

```
git status
```

Then:

```
git log --oneline -3
```

4. Create the v1.0 Tag

Step 2: Create an annotated tag

Run:

```
git tag -a v1.0 -m "Release version 1.0"
```

Step 3: Verify the tag

Run:

```
git tag
```

Check details:

```
git show v1.0
```

5. Push the Tag to GitHub

Step 4: Push v1.0

First check your remote:

```
git remote -v
```

If origin is configured, push:

```
git push origin v1.0
```

Step 5: Verify on GitHub

Go to your GitHub repository and open:

Releases → Tags

You should find:

V1.0

6. Delete the Tag Locally

Step 6: Delete local v1.0

Run:

```
git tag -d v1.0
```

7. Delete the Tag from GitHub

Step 7: Delete the remote tag

Run:

```
git push origin --delete v1.0
```

Verify the restored tag

Run:

```
git tag
```

Push the Restored Tag to GitHub

Run:

```
git push origin v1.0
```

github.com/asifshaik6558/git-undo-wrong-push

asifshaik6558 / git-undo-wrong-push

Type to search

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

git-undo-wrong-push

Public

Pin

Watch 0

Fork 0

Star 0

master

Go to file

Code

asifshaik6558

Revert "Added wrong file"

fd1e79e · 5 hours ago

3 Commits

msg.txt

initial commit

6 hours ago

README

Add a README

Help people interested in this repository understand your project.

Add a README

About

No description, website, or topics provided.

Activity

0 stars

0 watching

0 forks

Releases

1 tag

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

asifshaik6558

github.com/asifshaik6558/git-undo-wrong-push/tags

asifshaik6558 / git-undo-wrong-push

Type to search

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Releases

Tags

Tags

v1.0

3 minutes ago

ff70662

zip

tar.gz

© 2026 GitHub, Inc.

Terms

Privacy

Security

Status

Community

Docs

Contact

Manage cookies

Do not share my personal information

```

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git tag -a v1.0 -m "Release version 1.0"

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git tag
v1.0

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git show v1.0
tag v1.0
Tagger: Asif <asifshaik6558@gmail.com>
Date: Sun Aug 23 21:44:14 2026 +0530

Release version 1.0

commit ff70662517717e60ac9deef24b38f58fc5ecfcc (HEAD -> rebase-demo, tag: v1.0)
Author: Asif <asifshaik6558@gmail.com>
Date: Sun Aug 23 18:02:08 2026 +0530

    Added web page

    Added CSS

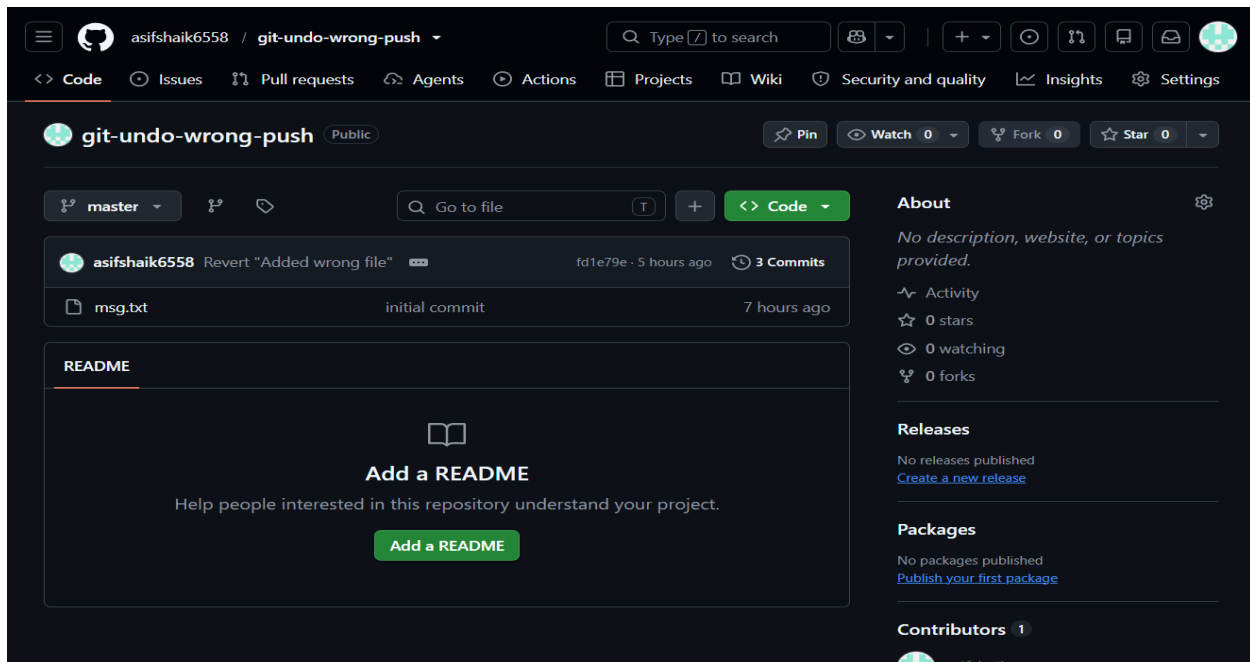
    Added footer

diff --git a/webpage.txt b/webpage.txt
new file mode 100644
index 0000000..ddd62f7
--- /dev/null
+++ b/webpage.txt
@@ -0,0 +1,3 @@
+HTML content
+CSS styling
+Footer section

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git remote -v
origin git@github.com:asifshaik6558/git-undo-wrong-push.git (fetch)
origin git@github.com:asifshaik6558/git-undo-wrong-push.git (push)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git push origin v1.0
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (11/11), 995 bytes | 199.00 KiB/s, done.
Total 11 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), done.
To github.com:asifshaik6558/git-undo-wrong-push.git
 * [new tag] v1.0 -> v1.0

```



```
asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git remote -v
origin  git@github.com:asifshaik6558/git-undo-wrong-push.git (fetch)
origin  git@github.com:asifshaik6558/git-undo-wrong-push.git (push)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git push origin v1.0
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (11/11), 995 bytes | 199.00 KiB/s, done.
Total 11 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), done.
To github.com:asifshaik6558/git-undo-wrong-push.git
 * [new tag]          v1.0 -> v1.0

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git tag -d v1.0
Deleted tag 'v1.0' (was 94ea071)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git push origin --delete v1.0
To github.com:asifshaik6558/git-undo-wrong-push.git
- [deleted]          v1.0

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git tag -a v1.0 ff70662 -m "Release version 1.0"

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git tag
v1.0

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git push origin v1.0
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (11/11), 995 bytes | 199.00 KiB/s, done.
Total 11 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), done.
To github.com:asifshaik6558/git-undo-wrong-push.git
 * [new tag]          v1.0 -> v1.0
```

13. Conclusion

In this challenge, a Git version tag named **v1.0** was created to mark a specific release commit. The tag was pushed to GitHub and then deleted both locally and remotely. Using the original commit ID, the v1.0 tag was recreated and pushed back to GitHub.

8. Clone with Sparse Checkout

Clone only a subdirectory of a repo using sparse checkout.

1. Purpose

The purpose of this challenge is to learn how to clone a Git repository while downloading only a specific subdirectory instead of the entire repository.

This is useful when a repository is very large but you only need one folder.

2. Objective

- Create or use a GitHub repository containing multiple folders.
- Clone the repository using sparse checkout.
- Configure Git to download only one required subdirectory.
- Verify that only the selected folder is visible in the working directory.

Step 1: Go to a different location

Don't clone inside your existing repository.

For example:

```
cd ~
```

Create a directory for the sparse checkout:

```
mkdir sparse-practice  
cd sparse-practice
```

Step 2: Clone the repository without checking out all files

Use:

```
git clone --no-checkout git@github.com:USERNAME/REPOSITORY.git
```

For example:

```
git clone --no-checkout git@github.com:asifshaik6558/devops-git-practice.git
```

Go into the repository:

```
cd devops-git-practice
```

4. Enable Sparse Checkout

Step 3: Enable sparse checkout

Run:

```
git sparse-checkout init --cone
```

Step 4: Select the required directory

Suppose you only want the linux folder.

Run:

```
git sparse-checkout set linux
```

5. Verify the Result

Step 5: Check the files

Run:

```
Ls
```

Check inside:

```
ls linux
```

Step 6: Check sparse checkout configuration

Run:

```
git sparse-checkout list
```

6. Check Git Status

Run:

```
git status
```

```

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ cd ~

asifs@Asif_Shaik MINGW64 ~
$ mkdir sparse-practice

asifs@Asif_Shaik MINGW64 ~
$ cd sparse-practice

asifs@Asif_Shaik MINGW64 ~/sparse-practice
$ git clone --no-checkout git@github.com:asifshaik6558/devops-git-practice.git
Cloning into 'devops-git-practice'...
remote: Enumerating objects: 26, done.
remote: Counting objects: 100% (26/26), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 26 (delta 10), reused 20 (delta 5), pack-reused 0 (from 0)
Receiving objects: 100% (26/26), done.
Resolving deltas: 100% (10/10), done.

asifs@Asif_Shaik MINGW64 ~/sparse-practice
$ cd devops-git-practice

asifs@Asif_Shaik MINGW64 ~/sparse-practice/devops-git-practice (main)
$ git sparse-checkout init --cone

asifs@Asif_Shaik MINGW64 ~/sparse-practice/devops-git-practice (main|SPARSE)
$ git sparse-checkout set linux

asifs@Asif_Shaik MINGW64 ~/sparse-practice/devops-git-practice (main|SPARSE)
$ ls

asifs@Asif_Shaik MINGW64 ~/sparse-practice/devops-git-practice (main|SPARSE)
$ git sparse-checkout list
linux

```

Conclusion

In this challenge, we cloned a Git repository using sparse checkout and configured Git to check out only a specific subdirectory.

The main commands are:

```

git clone --no-checkout <repository-url>
git sparse-checkout init --cone
git sparse-checkout set <directory>

```

Key concept

Sparse checkout allows you to work with only selected directories of a repository instead of having the entire repository appear in your working directory.

This is especially useful for large repositories, monorepos, and projects where you only need one particular component.

9. Reset vs Revert Challenge

Demonstrate the difference between `git reset --hard` and `git revert` in a repo.

1. Purpose

To understand the difference between `git reset --hard` and `git revert` when undoing Git commits.

2. Objective

- Demonstrate `git reset --hard`.
- Demonstrate `git revert`.
- Compare their effect on Git history.

3. Implementation

Part A: `git reset --hard`

Create and commit a file:

```
echo "Version 1" > reset.txt
```

```
git add reset.txt
```

```
git commit -m "Added version 1"
```

Make another commit:

```
echo "Version 2" >> reset.txt
```

```
git add reset.txt
```

```
git commit -m "Added version 2"
```

Check history:

```
git log --oneline -3
```

Undo the latest commit:

```
git reset --hard HEAD~1
```

Check:

```
git log --oneline -3
```

Added version 2 is no longer part of the current branch history.

Part B: git revert

Create another commit:

```
echo "Version 3" >> reset.txt  
git add reset.txt  
git commit -m "Added version 3"
```

Check:

```
git log --oneline -3
```

Undo it using:

```
git revert HEAD
```

Check history:

```
git log --oneline -4
```

4. Difference

git reset --hard	git revert
Moves branch backward	Creates a new commit
Removes commit from current branch history	Keeps original commit in history
Discards working-tree changes	Reverses the changes
Best for local/unpublished commits	Preferred for pushed/shared commits

```

asifs@Asif_Shaik MINGW64 ~
$ cd ~/git-merge-conflict

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ vi reset.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git add reset.txt
warning: in the working copy of 'reset.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git commit -m "Added version 1"
[rebase-demo 5be3913] Added version 1
1 file changed, 1 insertion(+)
create mode 100644 reset.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ vi reset.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git add reset.txt
warning: in the working copy of 'reset.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git commit -m "Added version 2"
[rebase-demo fa83aee] Added version 2
1 file changed, 1 insertion(+)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git log --oneline -3
fa83aee (HEAD -> rebase-demo) Added version 2
5be3913 Added version 1
ff70662 (tag: v1.0) Added web page

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git reset --hard HEAD~1
HEAD is now at 5be3913 Added version 1

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git log --oneline -3
5be3913 (HEAD -> rebase-demo) Added version 1
ff70662 (tag: v1.0) Added web page
34c8747 (master) Added login feature

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ vi reset.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git add reset.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git commit -m "Added version 3"
[rebase-demo a995ef5] Added version 3
1 file changed, 1 insertion(+)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git log --oneline -3
a995ef5 (HEAD -> rebase-demo) Added version 3
5be3913 Added version 1
ff70662 (tag: v1.0) Added web page

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git revert HEAD
[rebase-demo b7346bc] Revert "Added version 3"
1 file changed, 1 deletion(-)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git log --oneline -4
b7346bc (HEAD -> rebase-demo) Revert "Added version 3"
a995ef5 Added version 3
5be3913 Added version 1
ff70662 (tag: v1.0) Added web page

```

5. Conclusion

git reset --hard moves the branch back and removes the commit from the current branch history.

git revert creates a new commit that undoes the previous commit while preserving history.

Key commands

git reset --hard HEAD~1 git revert HEAD

10. Detached HEAD Challenge

Checkout a specific commit (detached HEAD state) and create a new branch from it.

1. Purpose

To understand Detached HEAD state and learn how to create a new branch from a specific commit.

2. Objective

- View the commit history.
- Checkout a specific commit.
- Enter Detached HEAD state.
- Create a new branch from that commit.
- Verify the new branch.

3. Implementation

Step 1: Check commit history

```
git log --oneline -5
```

Step 2: Checkout the specific commit

```
git checkout 34c8747
```

Check:

```
git status
```

Step 3: Create a branch from this commit

Run:

```
git checkout -b detached-feature
```

Step 4: Verify

Check the branch:

```
git branch
```

Check the commit:

`git log --oneline -3`

```
MINGW64:/c/Users/asifs/git-merge-conflict
asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git log --oneline -5
b7346bc (HEAD -> rebase-demo) Revert "Added version 3"
a995ef5 Added version 3
5be3913 Added version 1
ff70662 (tag: v1.0) Added web page
34c8747 (master) Added login feature

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-demo)
$ git checkout 34c8747
Note: switching to '34c8747'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 34c8747 Added login feature

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict ((34c8747...))
$ git status
HEAD detached at 34c8747
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    config.txt

nothing added to commit but untracked files present (use "git add" to track)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict ((34c8747...))
$ git checkout -b detached-feature
Switched to a new branch 'detached-feature'

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (detached-feature)
$ git branch
* detached-feature
  developer1
  developer2
  feature-login
  master
  rebase-demo
  recovery-test

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (detached-feature)
$ git log --oneline -3
34c8747 (HEAD -> detached-feature, master) Added login feature
f049ca2 Added application files
fd1e79e (origin/master) Revert "Added wrong file"
```

4. Conclusion

A specific commit was checked out, putting Git into Detached HEAD state. A new branch was then created from that commit using:

```
git checkout -b detached-feature
```

Key concept: Detached HEAD means HEAD points directly to a commit rather than a branch. Creating a branch from that commit gives the work a branch reference so it can be continued safely.

11. Git Hooks Challenge

Configure a pre-commit hook to reject commits without a message format (e.g., must start with JIRA-XXX).

1. Purpose

To configure a pre-commit hook that checks the commit message and rejects the commit if it does not start with the required format:

JIRA-XXX

Example of a valid message:

JIRA-123 Added login feature

2. Objective

- Create a pre-commit hook.
- Check the commit message format.
- Reject an invalid commit message.
- Allow a valid commit message.

Important: A pre-commit hook normally does not receive the commit message directly. For checking commit-message format, the correct Git hook is commit-msg. So we will use `.git/hooks/commit-msg`.

3. Implementation

Step 1: Go to your Git repository

```
cd ~/git-merge-conflict
```

Step 2: Create the commit-msg hook

Run:

```
vi .git/hooks/commit-msg
```

Step 3: Give the hook execute permission

```
chmod +x .git/hooks/commit-msg
```

4. Test Invalid Commit

Create a test file:

```
echo "Testing Git hook" > hook-test.txt
```

Stage it:

```
git add hook-test.txt
```

Try an invalid commit:

```
git commit -m "Added hook test"
```

5. Test Valid Commit

Now use the required format:

```
git commit -m "JIRA-123 Added hook test"
```

6. Verify

Run:

```
git log --oneline -2
```

Check the hook:

```
ls -l .git/hooks/commit-msg
```

```
MINGW64:/c/Users/asifs/git-merge-conflict
asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (detached-feature)
$ git branch
* detached-feature
  developer1
  developer2
  feature-login
  master
  rebase-demo
  recovery-test

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (detached-feature)
$ git log --oneline -3
34c8747 (HEAD -> detached-feature, master) Added login feature
f049ca2 Added application files
fd1e79e (origin/master) Revert "Added wrong file"

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (detached-feature)
$ cd ~/git-merge-conflict

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (detached-feature)
$ vi .git/hooks/commit-msg

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (detached-feature)
$ chmod +x .git/hooks/commit-msg

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (detached-feature)
$ touch hook-test.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (detached-feature)
$ vi hook-test.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (detached-feature)
$ git add hook-test.txt
warning: in the working copy of 'hook-test.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (detached-feature)
$ git commit -m "Added hook test"
ERROR: Commit message must start with JIRA-XXX
Example: JIRA-123 Added login feature

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (detached-feature)
$ git commit -m "JIRA-123 Added hook test"
[detached-feature b83993d] JIRA-123 Added hook test
1 file changed, 1 insertion(+)
create mode 100644 hook-test.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (detached-feature)
$ git log --oneline -2
b83993d (HEAD -> detached-feature) JIRA-123 Added hook test
34c8747 (master) Added login feature

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (detached-feature)
$ ls -l .git/hooks/commit-msg
-rwxr-xr-x 1 asifs 197609 213 Aug 23 23:38 .git/hooks/commit-msg*
```

7. Conclusion

A Git commit-msg hook was configured to validate commit messages before they are created.

Key command

```
chmod +x .git/hooks/commit-msg
```

Key concept

Git hooks automatically run scripts at specific Git operations. The commit-msg hook can be used to enforce a standard commit-message format.

12.Squash Merge vs Rebase Merge

Show the difference between squash merge and rebase merge with evidence.

1. Purpose

To understand the difference between Squash Merge and Rebase Merge and observe how each method affects Git history.

2. Objective

- Create multiple commits on a feature branch.
- Perform a Squash Merge and observe the history.
- Perform a Rebase Merge and observe the history.
- Compare the results with `git log --graph`.

3. Implementation

Part A — Squash Merge

Step 1: Create a feature branch

```
git checkout master  
git checkout -b squash-feature
```

Step 2: Create 3 commits

```
echo "Line 1" > squash.txt  
git add squash.txt  
git commit -m "Added line 1"echo "Line 2" >> squash.txt  
git add squash.txt  
git commit -m "Added line 2"echo "Line 3" >> squash.txt  
git add squash.txt  
git commit -m "Added line 3"
```

Check:

```
git log --oneline --graph
```

Step 3: Squash merge

Switch to master:

```
git checkout master
```

Run:

```
git merge --squash squash-feature
```

Then:

```
git commit -m "Added squash feature"
```

Check:

```
git log --oneline --graph
```

4. Rebase Merge

For a clean demonstration, create another feature branch from the current master:

```
git checkout -b rebase-feature
```

Create 3 commits:

```
echo "A" > rebase.txt
```

```
git add rebase.txt
```

```
git commit -m "Added A"echo "B" >> rebase.txt
```

```
git add rebase.txt
```

```
git commit -m "Added B"echo "C" >> rebase.txt
```

```
git add rebase.txt
```

```
git commit -m "Added C"
```

Check:

```
git log --oneline --graph
```

Step 5: Rebase the feature branch

First switch to the feature branch if you aren't already there:

```
git checkout rebase-feature
```

Rebase it onto master:

```
git rebase master
```

Then switch to master:

```
git checkout master
```

Fast-forward merge:

```
git merge --ff-only rebase-feature
```

Check the history:

```
git log --oneline --graph
```

You will see the individual commits:

Difference

Squash Merge	Rebase Merge
Combines multiple commits into one	Keeps individual commits
Creates one new commit on target branch	Replays commits onto new base
Simplifies history	Preserves detailed commit history
Good for many small/temporary commits	Good when individual commits are meaningful
git merge --squash	git rebase + fast-forward merge

MINGW64:/c/Users/asifs/git-merge-conflict

```
asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git checkout -b rebase-feature
Switched to a new branch 'rebase-feature'

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ touch rebase.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ vi rebase.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ git add rebase.txt
warning: in the working copy of 'rebase.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ git commit -m "JIRA-105 Added A"
[rebase-feature 51c4e30] JIRA-105 Added A
1 file changed, 1 insertion(+)
create mode 100644 rebase.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ vi rebase.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ git add rebase.txt
warning: in the working copy of 'rebase.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ git commit -m "JIRA-106 Added B"
[rebase-feature 08c6fb7] JIRA-106 Added B
1 file changed, 1 insertion(+)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ vi rebase.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ git add rebase.txt
warning: in the working copy of 'rebase.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ git commit -m "JIRA-107 Added C"
[rebase-feature 2d8e8f1] JIRA-107 Added C
1 file changed, 1 insertion(+)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ git log --oneline --graph -5
* 2d8e8f1 (HEAD -> rebase-feature) JIRA-107 Added C
* 08c6fb7 JIRA-106 Added B
* 51c4e30 JIRA-105 Added A
* 8ce066c (master) JIRA-104 Added squash feature
* 34c8747 Added login feature

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ git rebase master
Current branch rebase-feature is up to date.
```

```
asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ git log --oneline --graph -5
* 2d8e8f1 (HEAD -> rebase-feature) JIRA-107 Added C
* 08c6fb7 JIRA-106 Added B
* 51c4e30 JIRA-105 Added A
* 8ce066c (master) JIRA-104 Added squash feature
* 34c8747 Added login feature

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ git rebase master
Current branch rebase-feature is up to date.

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (rebase-feature)
$ git checkout master
Switched to branch 'master'
Your branch is ahead of 'origin/master' by 3 commits.
(use "git push" to publish your local commits)

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git merge --ff-only rebase-feature
Updating 8ce066c..2d8e8f1
Fast-forward
 rebase.txt | 3 +++
1 file changed, 3 insertions(+)
create mode 100644 rebase.txt

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git log --oneline --graph -6
* 2d8e8f1 (HEAD -> master, rebase-feature) JIRA-107 Added C
* 08c6fb7 JIRA-106 Added B
* 51c4e30 JIRA-105 Added A
* 8ce066c JIRA-104 Added squash feature
* 34c8747 Added login feature
* f049ca2 Added application files

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git log --oneline --graph -6
* 2d8e8f1 (HEAD -> master, rebase-feature) JIRA-107 Added C
* 08c6fb7 JIRA-106 Added B
* 51c4e30 JIRA-105 Added A
* 8ce066c JIRA-104 Added squash feature
* 34c8747 Added login feature
* f049ca2 Added application files
```

7. Conclusion

Squash merge combines all feature-branch changes into one clean commit.

Rebase merge keeps the individual commits but places them on top of the target branch, producing a linear history.

The main difference is:

Squash = one combined commit.

Rebase = individual commits preserved in a linear history.

13. Fork & Pull Request Workflow

Fork a repo, make a change, and submit a pull request to the original repo.

1. Purpose

To learn how to contribute to another GitHub repository using the Fork → Clone → Branch → Push → Pull Request workflow.

2. Objective

- Fork a repository.
- Clone your fork.
- Create a new branch.
- Make a change.
- Push the branch to your fork.
- Create a Pull Request to the original repository.

3. Implementation

Step 1: Fork a repository

Go to any GitHub repository you want to practice with.

On the repository page, click:

Fork → Create fork

After forking, you will have your own copy.

fork:

`github.com/asifshaik6558/demo-repo`

Step 2: Clone your fork

Copy the SSH URL of your fork and run:

```
git clone git@github.com:asifshaik6558/demo-repo.git
```

Then:

```
cd demo-repo
```

Check:

```
git remote -v
```

Step 3: Create a feature branch

```
git checkout -b feature-update
```

Check:

```
git branch
```

Step 4: Make a change

For example:

```
echo "Added contribution" > contribution.txt
```

Then:

```
git add contribution.txt
```

Commit:

```
git commit -m "JIRA-108 Added contribution"
```

Step 5: Push your branch

git push origin feature-update

Step 6: Create Pull Request

Go to your fork on GitHub.

You should see an option such as:

Compare & pull request

Click it.

Make sure the PR direction is:

YOUR FORK

feature-update



ORIGINAL REPOSITORY

main

For example:

asifshaik6558/demo-repo

feature-update



OWNER/demo-repo

main

Add a title:

JIRA-108 Added contribution

Add a description:

Added contribution file as part of Git Fork and Pull Request practice.

Click:

Create pull request

4. Verify

After creating the PR, GitHub should show:

Open

JIRA-108 Added contribution

The PR is now submitted to the original repository.

5. Workflow

Original Repository



Fork



Your Fork



Clone



Create Branch



Make Changes



Commit



Push



Pull Request



Original Repository

```

asifs@Asif_Shaik MINGW64 ~
$ git clone https://github.com/asifshaik6558/Hello-world.git
Cloning into 'Hello-World'...
remote: Enumerating objects: 7, done.
remote: Total 7 (delta 0), reused 0 (delta 0), pack-reused 7 (from 1)
Receiving objects: 100% (7/7), done.

asifs@Asif_Shaik MINGW64 ~
$ cd Hello-World

asifs@Asif_Shaik MINGW64 ~/Hello-World (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean

asifs@Asif_Shaik MINGW64 ~/Hello-World (master)
$ git checkout -b my-change
Switched to a new branch 'my-change'

asifs@Asif_Shaik MINGW64 ~/Hello-World (my-change)
$ ls
README

asifs@Asif_Shaik MINGW64 ~/Hello-world (my-change)
$ git checkout -b my-change
fatal: a branch named 'my-change' already exists

asifs@Asif_Shaik MINGW64 ~/Hello-world (my-change)
$ git branch
  master
* my-change

asifs@Asif_Shaik MINGW64 ~/Hello-world (my-change)
$ nano README

asifs@Asif_Shaik MINGW64 ~/Hello-world (my-change)
$ git status
On branch my-change
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README

no changes added to commit (use "git add" and/or "git commit -a")

```

```

no changes added to commit (use "git add" and/or "git commit -a")

asifs@Asif_Shaik MINGW64 ~/Hello-World (my-change)
$ git add README

asifs@Asif_Shaik MINGW64 ~/Hello-World (my-change)
$ git status
On branch my-change
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README

asifs@Asif_Shaik MINGW64 ~/Hello-World (my-change)
$ git commit -m "Updated README"
[my-change 10a23e4] Updated README
 1 file changed, 1 insertion(+)

asifs@Asif_Shaik MINGW64 ~/Hello-World (my-change)
$ git add README

asifs@Asif_Shaik MINGW64 ~/Hello-World (my-change)
$ git status
On branch my-change
nothing to commit, working tree clean

asifs@Asif_Shaik MINGW64 ~/Hello-World (my-change)
$ git log --oneline -3
10a23e4 (HEAD -> my-change) Updated README
7fd1a60 (origin/master, origin/HEAD, master) Merge pull request #6 from Spaceghost/patch-1
7629413 New line at end of file. --Signed off by Spaceghost

asifs@Asif_Shaik MINGW64 ~/Hello-World (my-change)
$ git status
On branch my-change
nothing to commit, working tree clean

asifs@Asif_Shaik MINGW64 ~/Hello-world (my-change)
$ git push origin my-change
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 326 bytes | 163.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'my-change' on GitHub by visiting:
remote:   https://github.com/asifshaik6558/Hello-world/pull/new/my-change
remote:
To https://github.com/asifshaik6558/Hello-world.git
 * [new branch]      my-change -> my-change

```

Browser tabs: asifshaik6558/Hello-World: My x Document 19.docx x + Ask Gemini | - □ x

Address bar: github.com/asifshaik6558/Hello-World/tree/master

Navigation: <> Code Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Repository: **Hello-World** Public
forked from [octocat/Hello-World](#)

Buttons: Pin Watch 0 Fork 0 Star 0

Branch: master Go to file + <> Code

This branch is up to date with octocat/Hello-World:master .
Contribute Sync fork

octocat Merge pull request octocat#6 from Spaceghost... 7fd1a60 · 15 years ago 3 Commits

README New line at end of file. --Signed off by ... 15 years ago

README
Hello World!

About
My first repository on GitHub!
Readme
Activity
0 stars
0 watching
0 forks

Releases
No releases published
[Create a new release](#)

Packages
No packages published
[Publish your first package](#)

Browser tabs: asifshaik6558/Hello-World at n x Document 19.docx x + Ask Gemini | - □ x

Address bar: github.com/asifshaik6558/Hello-World/tree/my-change

Navigation: <> Code Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Repository: **Hello-World** Public
forked from [octocat/Hello-World](#)

Buttons: Pin Watch 0 Fork 0 Star 0

Branch: my-change Go to file + <> Code

This branch is 1 commit ahead of octocat/Hello-World:master .
Contribute Sync fork

asifshaik6558 Updated README 10a23e4 · 6 minutes ago 4 Commits

README Updated README 6 minutes ago

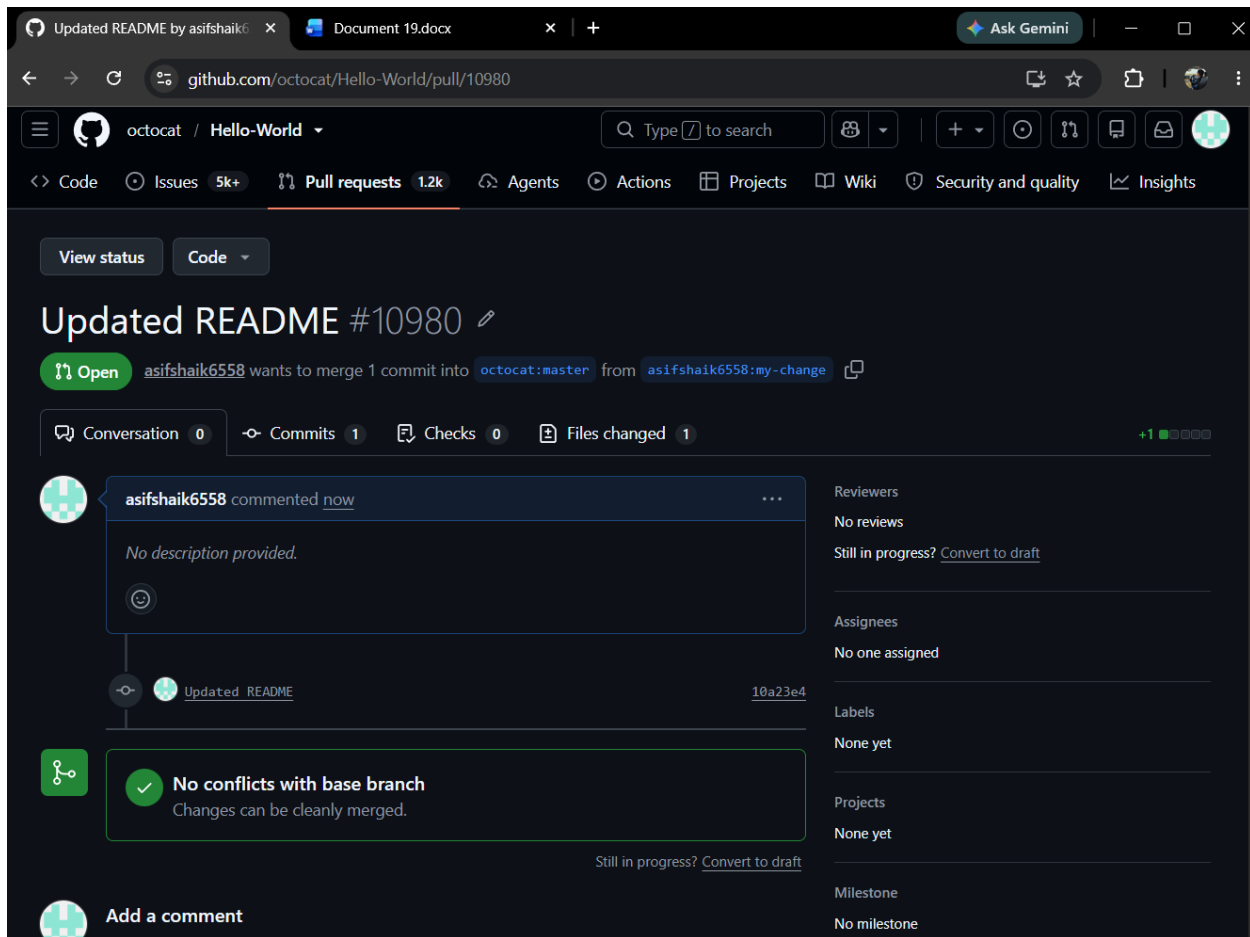
README
Hello World!
This change was made as part of my GitHub Fork and Pull Request practice.

About
My first repository on GitHub!
Readme
Activity
0 stars
0 watching
0 forks

Releases
No releases published
[Create a new release](#)

Packages
No packages published
[Publish your first package](#)

Contributors



6. Conclusion

The Fork & Pull Request workflow allows a developer to contribute to a repository without directly modifying the original repository.

14. Recover Lost Commit

Commit something, reset hard, and then recover it using `git reflog`.

Purpose

To learn how to recover a commit that was removed using `git reset --hard` by using `Git reflog`.

Objective

- Create a commit.
- Reset the branch to remove the commit.
- Find the lost commit using `git reflog`.
- Recover the commit.

Steps / Implementation

Step 1: Create a test file

```
echo "Important work" > lost.txt
```

Step 2: Commit it

```
git add lost.txt  
git commit -m "JIRA-109 Added important work"
```

Check the commit:

```
git log --oneline -3
```

Step 3: Reset hard

Run:

```
git reset --hard HEAD~1
```

Now the commit is no longer in the current branch history.

Check:

```
git log --oneline -3
```

The JIRA-109 commit should be gone.

Also:

ls

lost.txt should be gone.

Step 4: Find the lost commit

Run:

```
git reflog
```

Step 5: Recover the commit

Use the commit ID from your reflog:

```
git reset --hard abc1234
```

Or use the reflog reference directly:

```
git reset --hard HEAD@{1}
```

Step 6: Verify

```
git log --oneline -3
```

And:

Ls

MINGW64~/c/Users/asifs/git-merge-conflict

```
asifs@Asif-Shaik MINGW64 ~/git-merge-conflict (master)
$ touch lost.txt

asifs@Asif-Shaik MINGW64 ~/git-merge-conflict (master)
$ vi lost.txt

asifs@Asif-Shaik MINGW64 ~/git-merge-conflict (master)
$ git add lost.txt
warning: in the working copy of 'lost.txt', LF will be replaced by CRLF the next time Git touches it

asifs@Asif-Shaik MINGW64 ~/git-merge-conflict (master)
$ git commit -m "JIRA-109 Added important work"
[master 042195b] JIRA-109 Added important work
1 file changed, 1 insertion(+)
create mode 100644 lost.txt

asifs@Asif-Shaik MINGW64 ~/git-merge-conflict (master)
$ git log --oneline -3
042195b (HEAD -> master) JIRA-109 Added important work
2d8e8f1 (rebase-feature) JIRA-107 Added C
08c6fb7 JIRA-106 Added B

asifs@Asif-Shaik MINGW64 ~/git-merge-conflict (master)
$ git reset --hard HEAD~1
HEAD is now at 2d8e8f1 JIRA-107 Added C

asifs@Asif-Shaik MINGW64 ~/git-merge-conflict (master)
$ git log --oneline -3
2d8e8f1 (HEAD -> master, rebase-feature) JIRA-107 Added C
08c6fb7 JIRA-106 Added B
51c4e30 JIRA-105 Added A

asifs@Asif-Shaik MINGW64 ~/git-merge-conflict (master)
$ ls
app.txt  config.txt  congig.txt  login  msg.txt  rebase.txt  squash.txt
```

MINGW64~/c/Users/asifs/git-merge-conflict

```
app.txt  config.txt  congig.txt  login  msg.txt  rebase.txt  squash.txt

asifs@Asif-Shaik MINGW64 ~/git-merge-conflict (master)
$ git relog
2d8e8f1 (HEAD -> master, rebase-feature) HEAD@{0}: reset: moving to HEAD~1
042195b HEAD@{1}: commit: JIRA-109 Added important work
2d8e8f1 (HEAD -> master, rebase-feature) HEAD@{2}: merge rebase-feature: Fast-forward
8ce066c HEAD@{3}: checkout: moving from rebase-feature to master
2d8e8f1 (HEAD -> master, rebase-feature) HEAD@{4}: commit: JIRA-107 Added C
08c6fb7 HEAD@{5}: commit: JIRA-106 Added B
51c4e30 HEAD@{6}: commit: JIRA-105 Added A
8ce066c HEAD@{7}: checkout: moving from master to rebase-feature
8ce066c HEAD@{8}: commit: JIRA-104 Added squash feature
34c8747 HEAD@{9}: checkout: moving from squash-feature to master
251160f (squash-feature) HEAD@{10}: commit: JIRA-103 Added line-3
5b252ce HEAD@{11}: commit: JIRA-102 Added line-2
8a2ca33 HEAD@{12}: commit: JIRA-101 Added line-1
34c8747 HEAD@{13}: checkout: moving from master to squash-feature
34c8747 HEAD@{14}: checkout: moving from detached-feature to master
b83993d (detached-feature) HEAD@{15}: commit: JIRA-123 Added hook test
34c8747 HEAD@{16}: checkout: moving from 34c8747a9b7eb1f1ab1b8d1281c18ca7c5cb1ad to detached-feature
34c8747 HEAD@{17}: checkout: moving from rebase-demo to 34c8747
b7346bc (rebase-demo) HEAD@{18}: revert: Revert "Added version 3"
a995ef5 HEAD@{19}: commit: Added version 3
5be3913 HEAD@{20}: reset: moving to HEAD~1
fa83aee HEAD@{21}: commit: Added version 2
5be3913 HEAD@{22}: commit: Added version 1
ff70662 (tag: v1.0) HEAD@{23}: rebase (finish): returning to refs/heads/rebase-demo
ff70662 (tag: v1.0) HEAD@{24}: rebase (squash): Added web page
452b330 HEAD@{25}: rebase (squash): # This is a combination of 2 commits.
b141591 HEAD@{26}: rebase (start): checkout HEAD~3
039629b HEAD@{27}: commit: Added footer
6ccb3d8 HEAD@{28}: commit: Added CSS
b141591 HEAD@{29}: commit: Added web page
34c8747 HEAD@{30}: checkout: moving from master to rebase-demo
34c8747 HEAD@{31}: cherry-pick: Added login feature
f049ca2 HEAD@{32}: checkout: moving from feature-login to master
2cca90b (feature-login) HEAD@{33}: commit: Added login feature
f049ca2 HEAD@{34}: checkout: moving from master to feature-login
f049ca2 HEAD@{35}: commit (amend): Added application files
5c7d67c HEAD@{36}: commit: Added application files
fd1e79e (origin/master) HEAD@{37}: revert: Revert "Added wrong file"
ec7ccbc HEAD@{38}: commit: Added wrong file
a7d8036 HEAD@{39}: checkout: moving from recovery-test to master
2a3c8ac (recovery-test) HEAD@{40}: checkout: moving from master to recovery-test
a7d8036 HEAD@{41}: checkout: moving from recovery-test to master
2a3c8ac (recovery-test) HEAD@{42}: commit: Added recovery data
a7d8036 HEAD@{43}: checkout: moving from master to recovery-test
a7d8036 HEAD@{44}: checkout: moving from developer2 to master
503ab68 (developer2) HEAD@{45}: commit (merge): Resolved merge conflict
a965028 HEAD@{46}: commit: Updated message by horizon
a7d8036 HEAD@{47}: checkout: moving from master to developer2
a7d8036 HEAD@{48}: checkout: moving from developer1 to master
c9f7242 (developer1) HEAD@{49}: commit: Updated message by techie
a7d8036 HEAD@{50}: checkout: moving from master to developer1
a7d8036 HEAD@{51}: commit (initial): initial commit
```

```
asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git reset --hard 042195b
HEAD is now at 042195b JIRA-109 Added important work

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ git log --oneline -3
042195b (HEAD -> master) JIRA-109 Added important work
2d8e8f1 (rebase-feature) JIRA-107 Added C
08c6fb7 JIRA-106 Added B

asifs@Asif_Shaik MINGW64 ~/git-merge-conflict (master)
$ ls
app.txt  config.txt  config.txt  login  lost.txt  msg.txt  rebase.txt  squash.txt
```

Conclusion

The lost commit was successfully recovered using git reflog.

Key commands

git reset --hard HEAD~1

git reflog

git reset --hard <lost-commit-id>

Key concept: git reflog records previous positions of HEAD, allowing you to recover commits that are no longer visible in the normal git log.